

eWater Platform API — Technical Requirements & Gap Analysis

Gap Analysis & Endpoint Specification

Prepared for: eWater Engineering Team

Prepared by: eWater Monitor project team (api-server / MCP integration)

Date: July 8, 2026

Status: Proposed — for eWater platform engineering review

How to read this document

This is a gap analysis, not an implementation plan for our side. It compares what our `api-server` (REST + MCP) currently consumes from eWater's public APIs against what is actually available across the six Power BI data sources eWater's reporting stack draws from (`state` , `credits` , `assetcontroller` , `ewater_reporting` , `pulse` , `ewater_central`). Every proposed endpoint below is something **eWater's platform team** would build and host — our project would then be one of several consumers. Nothing here implies we are building these endpoints ourselves, standing up a new dashboard UI, or resolving PII-handling policy; PII-bearing fields are flagged inline so eWater's team can apply their own data-handling policy.

Each new REST endpoint proposal is paired with an MCP tool, per this project's existing convention of never letting the two surfaces diverge (see `ewater-insights.ts` in our codebase) — we are recommending eWater's own public API adopt the same shared-logic / dual-surface discipline we use internally, not asking eWater to integrate with our MCP server.

1. Executive Summary

1.1 What exists today

eWater exposes four API bases (`auth` , `query` , `state` , `command`) that our api-server consumes for: asset listing and detail, EWC device settings, tank height / inflow / voltage / flow-rate time series (eSense charts), raw packet log retrieval and decoding, dispense-volume calibration (LCF/preload) analysis, a coarse fleet health/fault snapshot, sensor-range auto-detection, and meter/settings write-back (reset tick accumulator, apply calibration). This is a genuinely capable data-access layer for **individual-asset, telemetry-level** questions — "what is this device doing right now," "what should its calibration be." We have built an MCP server on top of the same logic so external LLM clients (e.g. WhatsApp-based agents) can ask these questions in natural language, with a consistent pagination envelope across all list-shaped tool responses.

1.2 What the data dictionary reveals is missing

The Power BI data dictionary describes six live databases holding materially more than what any current eWater API surfaces. Concretely, **nothing in the current API exposes**: lifecycle/state-change history for an asset (`assetlifecyclehistory`), pre-aggregated downtime (`ewater_reporting` downtime tables), pre-aggregated multi-grain usage rollups (`state.public.ztrilist_generalwateruse_*` , 6 grains from hourly to all-time), the credits domain at all (`tagtopup` , `tagtopupstate` , wallet balances), per-country currency/config (`ewater_central` . "Country" / "Currency"), or any system-level supply-vs-disbursement comparison for non-revenue water (NRW) analysis. There is also **no alerting primitive of any kind** — no webhook, no subscription, no "what changed since I last asked" feed — so every consumer (including us) is left to poll full snapshots and diff them client-side, which does not scale past a handful of assets and cannot catch state changes that happen between polls.

1.3 Structure of this document

Section 2 covers each capability area as a **current-state vs. target-state** gap analysis. Section 3 is the concrete endpoint catalogue: one REST + one MCP tool per capability, with backing tables, response shape, caveats pulled directly from the data dictionary, and priority. Section 4 is a dedicated deep-dive on alerting and predictive maintenance, since that capability doesn't exist at all today and needs its own design (webhook vs. polling feed, threshold rule model, risk scoring). Section 5 covers API hygiene: versioning, deprecation policy, response envelope consistency, and pagination standardization across **all** list endpoints (not just the ones we happen to consume via MCP today).

1.4 Priority summary

Priority	Meaning	Count
P0	Blocks a capability area that already has a partial, unreliable implementation today (e.g. NRW, credit integrity) or is required to make alerting possible at all	6 endpoints
P1	High value, no current workaround, clearly requested capability area	7 endpoints
P2	Hardening / completeness — makes an existing area more usable at fleet scale	4 endpoints

1.5 Out of scope (explicitly)

- Implementing any of these endpoints ourselves against eWater's databases — this is a requirements spec **for** eWater engineering.
- Dashboard UI/UX design for surfacing this data.
- PII governance policy (retention, consent, export rights). We flag which proposed response fields carry PII (tag owner identity, location, phone) so eWater's team can apply whatever policy they choose — we take no position on what that policy should be.

2. Capability Area Gap Analysis

For each area: **Today**, **Gap**, **Why it matters**, and pointer to the concrete endpoints in Section 3.

2.1 Asset health & status

Today: `GET /api/Asset/AssetHealthStatus`, `AssetPowerStatus`, `AssetFlowStatus`, `AssetUsageStatus` (query API) give a live snapshot per asset. `GET /api/Entity/HealthSnapshots` / `FaultSnapshots` give a fleet-wide rollup, but only current totals — no trend, no history of when an asset flipped from healthy to unhealthy.

Gap: There is no history endpoint for health/lifecycle state. `state.public.asset` carries a current `assetlifecyclestate`, and `state.public.assetlifecyclehistory` (per the data dictionary) records every transition, but nothing in the API returns it. Similarly, `ewater_reporting`'s downtime tables (pre-aggregated per-asset downtime windows) have no API surface — a consumer wanting "how much downtime did asset X have last month" must

currently reconstruct it from raw connectivity polling, which the current API doesn't even expose as a time series (only `LastCommsDt`, a single point-in-time value).

Why it matters: Every operational dashboard question about reliability ("which assets are flapping," "how long was this asset down," "did it recover on its own or need intervention") requires this and has no current path except polling `AssetHealthStatus` forever and building your own history table — which is exactly the kind of duplicated, inconsistent client-side logic this document is trying to head off industry-wide.

→ See 3.1 (`GET /assets/{assetId}/lifecycle-history`), 3.2 (`GET /assets/{assetId}/downtime`), 3.3 (`GET /health/fleet-summary`).

2.2 Alerting & anomaly detection

Today: Nothing. `HealthSnapshots` / `FaultSnapshots` are pull-only aggregates with no delta/changed-since semantics and no way to be notified. Our own project has a local `alert_rules` table, but it stores **our own derived config** (e.g. detected sensor range) — it is not backed by any eWater-side alerting primitive, because none exists.

Gap: No webhook registration, no "assets changed since cursor X" feed, no threshold-rule concept anywhere in the eWater API surface.

Why it matters: This is the single biggest capability gap. Every consumer that wants timely awareness of a fault (battery critical, flow stopped, tank empty) is forced into inefficient polling of full snapshots, which (a) doesn't scale as fleet size grows and (b) can miss transient state changes that resolve between polls. See the dedicated proposal in Section 4.

→ See 4 in full, plus 3.4–3.6.

2.3 Predictive maintenance

Today: Nothing that combines signals. `AssetPowerStatus` does expose useful raw ingredients — `trendDirection`, `endOfDayDeclineConsecutiveDays`, `endOfDayDeclineTotalVoltageDrop`, `todayLowBatteryEventCount` — but there is no endpoint that turns these (plus flow-rate stability and connectivity-gap frequency) into a single "this asset is likely to fail soon" signal.

Gap: No risk-scoring endpoint, fleet-wide or per-asset. The raw ingredients exist but require a consumer to independently invent the same weighting/scoring logic, which we already had to partially do internally for calibration drift (`suggestedLcf`) — the platform is better positioned to do this once, centrally, than let every consumer reinvent it.

Why it matters: Turns "battery dropped 0.3V a day for 5 days" from a fact you'd have to notice yourself into a proactive "replace battery within 2 weeks" signal — this is the difference between reactive and predictive maintenance.

→ See 4.3 (risk scoring proposal) and 3.7 (`GET /assets/{assetId}/risk-score`), 3.8 (`GET /risk-scores` fleet-wide).

2.4 NRW (non-revenue water) / leak detection

Today: Our own calibration/gap analysis (`getCalibrationAnalysis`) infers likely meter drift from dispense-volume KDE vs. LCF/preload settings — but this is a **per-meter calibration** signal, not

a true NRW measurement. It cannot detect a leak between the supply meter and the tap, only a miscalibrated tap meter.

Gap: The data dictionary describes exactly the data needed for real NRW: eSense (`purpose = 'eSense'`) assets measure **bulk supply into a water system**, while tag-dispensing assets (via `generalwateruse / waterdisbursement`) measure **litres actually dispensed to households**. The difference between the two, at the water-system level, over the same time window, is the NRW signal — and no current API endpoint computes or exposes it. `Supply (All)` in Power BI is filtered to only 3 hardcoded eSense assets and isn't a general fleet-wide table either.

Why it matters: This is the highest-value analytical gap in the whole document — NRW is a named requirement, the data to compute it already exists in `state` , and it is currently only reachable by hand-building the same DAX-equivalent query dictionary engineers wrote once for Power BI. Two data-quality rules are mandatory for anyone implementing this (see 3.9's caveats): eSense readings must have spike-filtering applied (>300 L/min = sensor fault) and eSense assets must never be summed into the disbursement side.

→ See 3.9 (`GET /watersystems/{id}/nrw`), 3.10 (`GET /nrw/summary` fleet-wide).

2.5 Tag-flashing detection

Today: No detection logic anywhere, ours or eWater's. "Tag-flashing" (rapid re-presentation of a tag to re-trigger a preload/free allowance, or exploiting a `FlowPreloadThreshold` window) is a known integrity concern implied by the existence of `FlowPreloadThreshold / FlowPreloadCharge` settings and the credits-domain topup-state tables, but nothing correlates rapid repeat taps against a single `nfcid` to flag it.

Gap: No endpoint returns per-tag event cadence at a granularity fine enough to detect "same tag, same asset, <N seconds apart, repeated M times" patterns. The datalog packet stream (`GetLogsForAssetByReceivedDate`) has the raw events, but nfcid-level burst detection across the whole fleet is not something any consumer can do without pulling and decoding raw packets for every asset continuously.

Why it matters: This is a direct revenue-integrity issue (free/underpriced water via preload-window abuse) and currently invisible unless someone manually reviews packet logs asset-by-asset.

→ See 3.11 (`GET /tags/{nfcid}/flashing-events`), 3.12 (`GET /assets/{assetId}/tag-flashing-summary`).

2.6 Credit / topup integrity

Today: Zero API surface. The `credits` DSN (`tagtopup` , `tagtopupstate`) is not touched by any current eWater endpoint we consume, and the data dictionary explicitly flags `supertapposition` as an unreliable indicator of topup delivery status — the dictionary's own recommendation is to join through `tagtopupstate.state` instead, which confirms this is a known, not hypothetical, data-quality issue.

Gap: No way to (a) list topups with their true delivery state, (b) find "stuck" topups (state not in a terminal collected/deleted/erased state, aged past a threshold), or (c) reconcile a tag's expected balance against topups-in minus usage-out.

Why it matters: Every topup that silently fails to reach the physical tag is a customer-facing trust problem ("I paid but got no water") that is currently undetectable except by the customer complaining. This is the second-highest-priority gap after alerting, because — like NRW — the fix is "expose data that already exists," not "invent new instrumentation."

→ See 3.13 (`GET /topups`), 3.14 (`GET /topups/stuck`), 3.15 (`GET /tags/{nfcid}/reconciliation`).

2.7 Multi-country currency / config

Today: Per-asset `priceOfWater` is derived from EWC device settings (`CurrencyConversion` , `LitresConversion` , `FlowConversion`) — this is asset-local and correct as far as it goes. But nothing in the API surfaces **country-level** currency config (currency code, exchange rate, average household size) — a consumer wanting to convert credits to a local currency, or estimate beneficiary counts per household, has no API to call and, per the data dictionary, the existing Power BI model hardcodes Kenya's values (`0.2109` KES/credit, `4.7` people/household) as if universal, which is simply wrong for the five other countries confirmed live (Gambia, UK, Portugal, Tanzania, Ghana).

Gap: No `GET` for `ewater_central."Country" / "Currency"` — `CreditExchangeRate` and `AvgPeoplePerHousehold` per country.

Why it matters: Any multi-country rollup (revenue, beneficiary counts) computed without this is currently silently wrong outside Kenya. This is a correctness bug waiting to be inherited by every future non-Kenya deployment, not just a missing convenience endpoint.

→ See 3.16 (`GET /countries/{countryId}/config`), 3.17 (`GET /currency-config` fleet-wide).

2.8 Asset lifecycle & uptime

Today: Current asset detail returns a single `assetLifecycleState` snapshot (`PreInstallation` / `Staged` / `Active` / etc.) — no history of transitions, no uptime/downtime percentage over a period.

Gap: Same underlying gap as 2.1 — `assetlifecyclehistory` exists in `state` but has no API. Distinct from 2.1 in that this is specifically about the **install-to-decommission lifecycle** (when did this asset go live, when was it staged, when was it decommissioned) rather than transient online/offline health flapping — both matter, for different questions (fleet growth/attrition reporting vs. operational reliability).

→ Same endpoints as 3.1/3.2 serve this; see caveats there for how the two lifecycle concepts (state-machine lifecycle vs. connectivity health) are distinguished in the response shape.

2.9 Fleet-wide query & pagination

Today: Inconsistent. Our MCP tool layer added a pagination envelope (`totalCount` / `returnedCount` / `offset` / `limit` / `hasMore`) specifically because `list_assets` was originally unbounded and could push an LLM client past its context window — but that fix was applied **only at our MCP wrapper layer**, on top of REST endpoints (`GetAssetList` , `Entity/List` , etc.) that remain themselves unpaginated at the eWater API level. Every consumer of

the raw eWater REST API (not just us) hits this same unbounded-response problem independently.

Gap: No consistent, platform-level pagination contract on eWater's own list endpoints. Consumers currently either fetch everything (fine at small scale, breaks as fleet grows past a few hundred assets) or invent their own paging wrapper as we did.

Why it matters: This isn't a new capability request, it's an API-hygiene fix that prevents every future consumer from re-solving the same problem we already had to solve once. See Section 5.3 for the concrete standard we recommend.

3. Proposed Endpoints

All proposed endpoints follow one shared list-response envelope (see 5.3) — `totalCount`, `returnedCount`, `offset`, `limit`, `hasMore` — mirroring the convention our own MCP layer already uses, applied here at the true API level instead of a wrapper. Base path assumed: `/api/v2/...` (see Section 5.1 for why a version prefix is proposed).

Format per endpoint: **Method & path** · **Purpose** · **Params** · **Response shape** · **Backing table(s)** · **Data caveats** · **Priority** · **Paired MCP tool**.

3.1 Asset lifecycle history

- **GET** `/api/v2/assets/{assetId}/lifecycle-history`
- **Purpose:** Return the full sequence of lifecycle-state transitions for one asset (install → staged → active → decommissioned, etc.), for fleet-growth/attrition reporting and "when did this asset actually go live" questions.
- **Params:** `assetId` (path, required); `from`, `to` (ISO date, optional, default all-time); `offset`, `limit` (default 50, max 500).
- **Response shape:**

```
{
  "assetId": 2706,
  "events": [
    { "state": "Active", "transitionDt": "2025-03-11T08:00:00Z",
      "previousState": "Staged" }
  ],
  "totalCount": 4, "returnedCount": 4, "offset": 0, "limit": 50, "hasMore":
false
}
```

- **Backing table:** `state.public.assetlifecyclehistory` (joined to `state.public.asset` for current state cross-check).
- **Data caveats:** None flagged in the dictionary beyond standard row-level integrity; confirm sort order is transition-time ascending so `previousState` chaining is unambiguous.
- **Priority:** P1
- **MCP tool:** `get_asset_lifecycle_history(assetId, from?, to?, offset?, limit?)`

3.2 Asset downtime windows

- **GET** `/api/v2/assets/{assetId}/downtime`
- **Purpose:** Pre-aggregated downtime windows (start, end, duration) for an asset over a period — the reliability-reporting counterpart to 3.1's lifecycle history.
- **Params:** `assetId` (path); `from`, `to` (default: trailing 30 days); `offset`, `limit`.
- **Response shape:**

```
{
  "assetId": 2706,
  "downtimeWindows": [
    { "startDt": "...", "endDt": "...", "durationMinutes": 132, "reason":
"connectivity" }
  ],
  "uptimePercent": 98.4,
  "totalCount": 6, "returnedCount": 6, "offset": 0, "limit": 50, "hasMore":
false
}
```

- **Backing table:** `ewater_reporting` downtime aggregation tables (per data dictionary `AssetDowntime`-family tables — exact table name to be confirmed with eWater's reporting team, as the dictionary describes this as a reporting-layer aggregate rather than a single named source table).
- **Data caveats:** Confirm whether `ewater_reporting`'s downtime tables already apply any minimum-gap threshold (e.g. ignore sub-1-minute blips) — if so, document the threshold in the response so consumers don't double-filter or misinterpret precision.
- **Priority:** P1
- **MCP tool:** `get_asset_downtime(assetId, from?, to?, offset?, limit?)`

3.3 Fleet health summary (paginated, per-asset)

- **GET** `/api/v2/health/fleet-summary`
- **Purpose:** Per-asset current health rollup at fleet scale with filtering — the paginated, per-asset-detail counterpart to today's aggregate-only `HealthSnapshots` / `FaultSnapshots`.
- **Params:** `status` (healthy/unhealthy/unknown), `waterSystemId`, `organisationId`, `countryId`, `offset`, `limit`.
- **Response shape:** `{ assets: [{ assetId, name, connectivityStatus, powerStatus, flowStatus, tankHeightStatus, lastUpdatedDt }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** Same source as `query.HealthSnapshots` / `FaultSnapshots`, exposed per-asset instead of pre-aggregated only.
- **Data caveats:** None beyond standard filtering semantics.
- **Priority:** P2
- **MCP tool:** `get_fleet_health_summary(status?, waterSystemId?, organisationId?, countryId?, offset?, limit?)`

3.4 Alert rules (CRUD)

- **POST** `/api/v2/alert-rules` / **GET** `/api/v2/alert-rules` / **PATCH** `/api/v2/alert-rules/{id}` / **DELETE** `/api/v2/alert-rules/{id}`

- **Purpose:** Let a consumer register threshold-based alert rules server-side (e.g. "battery < 3.2V", "no comms for > 24h") instead of re-polling and re-deriving thresholds client-side. Full design in Section 4.2.
- **Params (POST body):** `scope` (`asset` / `waterSystem` / `organisation` / `country` / `fleet`), `scopeId` , `metric` (enum, see 4.2), `operator` (`lt` / `gt` / `eq` / `noDataFor`), `threshold` , `windowMinutes` (for rate-of-change rules), `label` .
- **Response shape:** `{ id, scope, scopeId, metric, operator, threshold, windowMinutes, label, createdDt, enabled }`
- **Backing table:** New — this is a platform-side rules table eWater would need to create; no dictionary table currently models it.
- **Data caveats:** N/A (new capability).
- **Priority:** P0
- **MCP tools:** `create_alert_rule(...)` , `list_alert_rules(scope?, scopeId?, offset?, limit?)` , `update_alert_rule(id, ...)` , `delete_alert_rule(id)`

3.5 Active alerts feed

- **GET /api/v2/alerts**
- **Purpose:** List currently-fired alerts (rule breaches), filterable and paginated — the read side of 3.4, and the thing a webhook (4.1) would notify about.
- **Params:** `status` (`active` / `acknowledged` / `resolved`), `assetId` , `ruleId` , `since` (cursor), `offset` , `limit` .
- **Response shape:** `{ alerts: [{ id, ruleId, assetId, metric, firedDt, resolvedDt, status, value, threshold }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** New alert-events table (platform-side), populated by the rule evaluator described in 4.2.
- **Data caveats:** N/A (new capability).
- **Priority:** P0
- **MCP tool:** `get_active_alerts(status?, assetId?, ruleId?, since?, offset?, limit?)`

3.6 Health-change feed (polling cursor)

- **GET /api/v2/health/feed?since={cursor}**
- **Purpose:** "What changed since I last asked" — a cheap polling alternative for consumers who can't or won't run a webhook receiver. Full design in Section 4.1.
- **Params:** `since` (opaque cursor token, required after first call), `limit` .
- **Response shape:** `{ changes: [{ assetId, factor, previousStatus, newStatus, changedDt }], nextCursor, hasMore }`
- **Backing table:** Derived from the same source feeding `HealthSnapshots` , diffed server-side and buffered in a short-retention change log.
- **Data caveats:** N/A (new capability); document retention window (e.g. cursor expires after 24h of inactivity, forcing a client to fall back to a full snapshot).
- **Priority:** P0
- **MCP tool:** `get_health_change_feed(since?, limit?)`

3.7 Per-asset risk score

- **GET /api/v2/assets/{assetId}/risk-score**

- **Purpose:** Single predictive-maintenance signal (0–100) combining battery-decline trend, flow-anomaly frequency, and connectivity-gap frequency. Full model in 4.3.
- **Params:** `assetId` (path).
- **Response shape:** `{ assetId, riskScore, riskBand, contributingFactors: [{ factor, weight, detail }], computedDt }`
- **Backing table:** Computed from `query.AssetPowerStatus / AssetFlowStatus` history + downtime windows (3.2) — requires those to be persisted with history first, which is why this is sequenced after 3.1–3.3 in practice even though independently specified.
- **Data caveats:** Model needs a minimum history window (recommend ≥ 14 days of data) before a score is meaningful — return `riskScore: null, reason: "insufficient_history"` rather than a misleadingly confident low score for new assets.
- **Priority:** P1
- **MCP tool:** `get_asset_risk_score(assetId)`

3.8 Fleet-wide risk scores

- **GET** `/api/v2/risk-scores`
- **Purpose:** Paginated, sorted-by-risk-descending list for triage ("show me the 20 riskiest assets right now").
- **Params:** `waterSystemId?`, `organisationId?`, `countryId?`, `minRiskScore?`, `offset`, `limit` (default sort: `riskScore desc`).
- **Response shape:** `{ assets: [{ assetId, name, riskScore, riskBand }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** Same as 3.7, fleet-scanned.
- **Data caveats:** Same insufficient-history caveat as 3.7 — exclude or clearly flag assets below the minimum-history threshold rather than silently omitting them (a consumer paging through results should be able to tell "excluded because new" from "genuinely zero risk").
- **Priority:** P1
- **MCP tool:** `list_risk_scores(waterSystemId?, organisationId?, countryId?, minRiskScore?, offset?, limit?)`

3.9 Water-system NRW analysis

- **GET** `/api/v2/watersystems/{waterSystemId}/nrw`
- **Purpose:** Compare bulk supply (eSense) into a water system against actual metered disbursement to tags/households over the same window — the real NRW percentage.
- **Params:** `waterSystemId` (path); `from`, `to` (default trailing 30 days).
- **Response shape:**

```
{
  "waterSystemId": 41,
  "supplyLitres": 128000,
  "disbursedLitres": 96400,
  "nrwLitres": 31600,
  "nrwPercent": 24.7,
  "supplyAssetIds": [2471],
  "excludedSpikeLitres": 2100,
  "dataQuality": "supply_partial"
}
```

- **Backing table:** Supply side from `state.public.ztrilist_supplybyasset_0of6` (`purpose = 'eSense'` assets only); disbursement side from `state.public.ztrilist_generalwateruse_2of6_perday` joined through `wateruserid` → `asset` / `watersystem` (per data dictionary §4.4/4.5 worked examples).
- **Data caveats (mandatory, confirmed live 2026-06-18):**
 - **Exclude eSense from the disbursement side entirely** (`purpose <> 'eSense'`) — an eSense asset logged ~17M litres in one faulty session that would otherwise inflate a naive total by >1000x.
 - **Spike-filter the supply side:** discard any reading implying >300 L/min (18,000 L/hr) before summing — these are sensor-fault artifacts, not real supply. Report `excludedSpikeLitres` so consumers can see how much was filtered.
 - **Not every water system has an eSense supply meter** — `Supply (All)` today is filtered to only 3 hardcoded assets (2015, 2211, 2105) system-wide; `dataQuality` should reflect `"supply_partial"` / `"supply_unavailable"` when the water system in question lacks bulk-supply instrumentation, rather than silently returning `nrwPercent: null` with no explanation.
 - **Pre-cutover data caveat:** `generalwateruse` rollups are only reliably populated from ~July 2025 onward (the "Data Waterfall" cutover) — requests for `from` dates before that should either fall back to the legacy `waterdisbursement` path or return an explicit `dataQuality: "pre_cutover_unavailable"` rather than a silently-zero disbursement figure.
 - Also drop the ~0.003% of `generalwateruse` rows carrying sentinel placeholder timestamps (`2000-01-01` / `2086-01-01`) before aggregating.
- **Priority:** P0
- **MCP tool:** `get_nrw_analysis(waterSystemId, from?, to?)`

3.10 Fleet-wide NRW summary

- **GET** `/api/v2/nrw/summary`
- **Purpose:** Paginated NRW-percent ranking across all water systems with supply instrumentation — triage view for "which systems are leaking the most."
- **Params:** `countryId?` , `minNrwPercent?` , `offset` , `limit` (default sort: `nrwPercent desc`).
- **Response shape:** `{ waterSystems: [{ waterSystemId, name, nrwPercent, dataQuality }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** Same as 3.9, fleet-scanned; only includes systems where `dataQuality` is not `"supply_unavailable"` .
- **Data caveats:** Same as 3.9; additionally, systems with `supply_unavailable` should be countable (`totalSystemsCount` vs. `instrumentedSystemsCount`) so a consumer knows the denominator, not just the ranked list.

- **Priority:** P1
- **MCP tool:** `list_nrw_summary(countryId?, minNrwPercent?, offset?, limit?)`

3.11 Tag-flashing events

- **GET** `/api/v2/tags/{nfcid}/flashing-events`
- **Purpose:** Detect and list suspicious rapid repeat-presentation patterns for one tag (same tag, same asset, inter-arrival time below a configurable threshold, repeated beyond a configurable count) that suggest deliberate preload-window exploitation.
- **Params:** `nfcid` (path); `from`, `to`; `maxIntervalSeconds` (default: tie to the asset's own `FlowPreloadThreshold` setting, not a hardcoded global); `minRepeats` (default 3).
- **Response shape:** `{ nfcid, events: [{ assetId, burstStartDt, burstEndDt, tapCount, totalLitresPreloadOnly }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** Datalog packet stream per asset (event types 0x09/0x0B), keyed by `nfcid` from the packet's card UID field, cross-referenced with each asset's `FlowPreloadThreshold` / `FlowPreloadCharge` EWC settings.
- **Data caveats:** Threshold should be **per-asset**, not a single fleet-wide constant — `FlowPreloadThreshold` already varies per device per the EWC settings model our own `getAssetEwcSettings` already reads; reusing a wrong global constant here would under/over-flag depending on device config. **PII note:** `nfcid` is a pseudonymous tag identifier, not directly a person's identity, but if any response ever joins through to tag-owner records, that join introduces PII and should be flagged by eWater's own data-handling policy at that point — this endpoint's `nfcid`-keyed shape as specified does not itself add PII beyond what's already visible in tag records.
- **Priority:** P1
- **MCP tool:** `get_tag_flashing_events(nfcid, from?, to?, maxIntervalSeconds?, minRepeats?)`

3.12 Asset-level tag-flashing summary

- **GET** `/api/v2/assets/{assetId}/tag-flashing-summary`
- **Purpose:** Fleet-triage counterpart to 3.11 — "which assets have the most flashing-pattern activity," without requiring a consumer to already know which `nfcid`s to check.
- **Params:** `assetId` (path); `from`, `to`.
- **Response shape:** `{ assetId, distinctTagsFlagged, totalFlashingEvents, estimatedLitresLost }`
- **Backing table:** Aggregation of 3.11 at the asset level.
- **Data caveats:** Same threshold caveat as 3.11; `estimatedLitresLost` should state its assumption (e.g. preload-charge litres × event count) explicitly in a `methodology` field so it isn't mistaken for a precisely metered figure.
- **Priority:** P2
- **MCP tool:** `get_tag_flashing_summary(assetId, from?, to?)`

3.13 Topup listing

- **GET** `/api/v2/topups`
- **Purpose:** List wallet topups with their **true delivery state**, replacing the unreliable `supertapposition` field with a join through `tagtopupstate.state`.
- **Params:** `nfcid?`, `state?` (delivery state enum), `from?`, `to?`, `offset`, `limit`.

- **Response shape:** `{ topups: [{ id, nfcid, amount, currencyCode, requestedDt, deliveryState, deliveredDt }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** `credits.public.tagtopup` joined to `credits.public.tagtopupstate`.
- **Data caveats:** **Do not surface or derive delivery status from `supertaposition`** — the data dictionary explicitly confirms it is unreliable for this purpose; `tagtopupstate.state` is the source of truth. **PII note:** if `tagtopup` carries a purchaser phone number or payment reference, flag those fields explicitly in the eWater-side schema for their own PII policy — we take no position on whether/how they should be exposed here, only that they must be flagged, not silently included.
- **Priority:** P0
- **MCP tool:** `list_topups(nfcid?, state?, from?, to?, offset?, limit?)`

3.14 Stuck topups

- **GET `/api/v2/topups/stuck`**
- **Purpose:** Fleet-wide list of topups whose delivery state has not reached a terminal state (`collected` / `deleted` / `erased`) within a configurable age threshold — direct detection of the "customer paid, tag never got credit" failure mode.
- **Params:** `minAgeHours` (default 24), `offset`, `limit`.
- **Response shape:** `{ stuckTopups: [{ id, nfcid, amount, requestedDt, deliveryState, ageHours }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** Same as 3.13, filtered.
- **Data caveats:** Same `supertaposition` warning as 3.13; confirm the full terminal state enum with eWater engineering before finalizing (`collected` / `deleted` / `erased` are the three named in the dictionary, but there may be transient intermediate states that should not count as "stuck" — e.g. a topup 2 minutes old in a normal in-flight state should not appear here even though it isn't yet terminal).
- **Priority:** P0
- **MCP tool:** `get_stuck_topups(minAgeHours?, offset?, limit?)`

3.15 Tag balance reconciliation

- **GET `/api/v2/tags/{nfcid}/reconciliation`**
- **Purpose:** For one tag, compare expected balance (sum of delivered topups minus metered usage-credit-spend) against whatever the platform believes the tag's current balance to be, surfacing any discrepancy.
- **Params:** `nfcid` (path); `from?` (default: tag lifetime).
- **Response shape:** `{ nfcid, totalToppedUp, totalDelivered, totalUsageSpend, expectedBalance, reportedBalance, discrepancy, methodology }`
- **Backing table:** `credits.tagtopup` / `tagtopupstate` for topup side; usage-credit-spend from `state.public.ztrilist_generalwateruse_*` (post-cutover) — `(maximumstartcredit - minimumendcredit)/1000` per the data dictionary's documented formula — falling back to legacy `assetcontroller.public.waterdisbursement` for pre-cutover history.
- **Data caveats:** Must apply the same July-2025 cutover caveat as 3.9 — a naive lifetime sum will undercount pre-cutover usage-spend unless the legacy fallback path is actually implemented, not just documented. **PII note:** flag tag-owner identity if joined in by eWater's schema.
- **Priority:** P1
- **MCP tool:** `get_tag_reconciliation(nfcid, from?)`

3.16 Country currency/config

- **GET** `/api/v2/countries/{countryId}/config`
- **Purpose:** Expose the real per-country currency and demographic config that today is hardcoded as Kenya-only constants in Power BI DAX.
- **Params:** `countryId` (path).
- **Response shape:** `{ countryId, countryName, currencyCode, creditExchangeRate, avgPeoplePerHousehold }`
- **Backing table:** `ewater_central.public."Country"` joined to `ewater_central.public."Currency"` ; `CountryID` matches `state.public.country.id` directly (confirmed in dictionary — no fuzzy join needed).
- **Data caveats:** At least one row is known-bad in the source data (Austria's row maps to GMD with rate `1.0` — "clearly junk/test config, not a real rate," per the dictionary) — either exclude/flag known-invalid country config rows or let eWater engineering clean the source row; don't let this endpoint silently propagate obviously wrong test data to consumers as if it were real.
- **Priority:** P0
- **MCP tool:** `get_country_config(countryId)`

3.17 Fleet-wide currency/config listing

- **GET** `/api/v2/currency-config`
- **Purpose:** Bulk listing (all countries) for consumers building multi-country revenue or beneficiary-count rollups without one call per country.
- **Params:** `offset` , `limit` .
- **Response shape:** `{ countries: [{ countryId, countryName, currencyCode, creditExchangeRate, avgPeoplePerHousehold }], totalCount, returnedCount, offset, limit, hasMore }`
- **Backing table:** Same as 3.16.
- **Data caveats:** Same known-bad-row caveat as 3.16.
- **Priority:** P1
- **MCP tool:** `list_currency_config(offset?, limit?)`

4. Alerting & Predictive Maintenance — Dedicated Proposal

This capability doesn't exist today in any form (Section 2.2), so it needs a self-contained design rather than a single endpoint spec.

4.1 Health-change delivery: webhook + polling feed, not either/or

Recommend supporting **both** delivery modes from day one, because consumer infrastructure varies (some can run a public HTTPS receiver, some can only poll on a schedule):

- **Webhook mode** (`POST /api/v2/webhooks` to register a callback URL + event-type filter + shared secret for signature verification). eWater's platform pushes a JSON payload on each qualifying event (alert fired, alert resolved, health-factor transition). Must support:

exponential backoff retry on non-2xx, a way to list delivery failures per subscription, and HMAC signing so receivers can verify authenticity.

- **Polling feed mode** (3.6, `GET /health/feed?since=cursor`) for consumers without a webhook receiver, including cases like our own MCP server which is inherently request/response and stateless per invocation (see our architecture note: "no server-initiated notifications... needed for a tool-only server") — an MCP client can't receive a push at all, so the cursor-based feed is the *only* viable mode for MCP-style consumers, which is why 3.6 is specified as P0 alongside the webhook, not instead of it.

Both modes should be backed by the **same underlying change-detection log**, so there is one source of truth for "what changed and when," not two independently-implemented detection paths that could disagree.

4.2 Threshold alert rules — proposed model

A rule (3.4) has: `scope` (single asset up to whole fleet), `metric`, `operator`, `threshold`, optional `windowMinutes` for rate-of-change or no-data rules. Proposed initial metric enum, chosen to map directly onto fields the API already computes somewhere today (so the evaluator has no new instrumentation to build, only a rules engine watching existing fields):

Metric	Source field today	Example rule
<code>batteryVoltage</code>	<code>AssetPowerStatus.lastKnownVoltage</code>	<code>lt 3.2</code>
<code>batteryDeclineDays</code>	<code>AssetPowerStatus.endOfDayDeclineConsecutiveDays</code>	<code>gt 5</code>
<code>noCommsMinutes</code>	derived from <code>lastCommsDt</code>	<code>gt 1440</code> (24h)
<code>flowRateToday</code>	<code>AssetFlowStatus.todayAverageFlowRate</code>	<code>eq 0</code> combined with recent usage expectation
<code>tankHeightPercent</code>	<code>GetTankHeightHistoryByDateRange</code>	<code>lt 10</code>
<code>nrvPercent</code> (3.9)	new	<code>gt 30</code>
<code>riskScore</code> (3.7)	new	<code>gt 75</code>

Evaluation cadence should match each metric's natural update frequency (no point evaluating `batteryVoltage` more often than the device reports it) — recommend the evaluator be event-driven off ingestion, not a fixed global poll interval, to avoid needless load and to catch state changes immediately rather than up to one poll interval late.

4.3 Risk scoring — proposed model

Rather than a black-box score, recommend an explainable weighted model returning contributing factors (3.7's `contributingFactors` field), e.g.:

- **Battery trend factor:** normalized `endOfDayDeclineConsecutiveDays` and `endOfDayDeclineTotalVoltageDrop` — a battery declining steadily for 5+ days is a stronger signal than a single low reading.
- **Flow anomaly factor:** frequency of days where `todayAverageFlowRate` deviates sharply (e.g. >2 standard deviations) from the asset's own trailing baseline — asset-relative, not a

fleet-wide constant, since normal flow varies hugely by tap location/usage pattern.

- **Connectivity factor:** downtime-window frequency/duration from 3.2 over a trailing window (e.g. 30 days) — frequent short outages can be as predictive of impending failure as one long outage.

Each factor should be independently inspectable (`contributingFactors[].weight` and `.detail`) so a maintenance team can see *why* an asset scored high, not just the number — this is the same "surface the reasoning, not just a magic value" principle already applied in our own calibration-analysis feature (`getCalibrationAnalysis` 's plain-language interpretation), extended to the platform level.

Explicitly flag `riskScore: null, reason: "insufficient_history"` for assets without enough trailing data (recommend ≥ 14 days) rather than emitting a falsely-confident low score — a brand-new asset should read as "unknown," not "healthy."

5. Versioning, Deprecation & API Hygiene

5.1 Versioning

None of the current eWater API bases (`auth` , `query` , `state` , `command`) carry any version prefix or header. Recommend introducing `/api/v2/...` for every endpoint proposed in this document (the "v2" label signals "the next generation of this surface," not that today's endpoints are being renamed) — the existing unversioned endpoints continue to serve as-is; there is no requirement here to touch them. Establishing versioning now, before consumers proliferate further, avoids a much more painful retrofit later.

5.2 Response envelope consistency

Today, error responses vary base-to-base — some return a bare error string, some a structured object. Recommend a single error envelope for all new (`v2`) endpoints:

```
{ "error": { "code": "ASSET_NOT_FOUND", "message": "No asset with id 99999",  
  "details": {} } }
```

with a stable `code` enum documented per endpoint, so consumers can branch on `code` rather than string-matching `message` (which is exactly the kind of brittle client behavior a documented error taxonomy prevents).

5.3 Pagination — one contract for every list endpoint

Adopt, at the true API level, the same envelope our MCP layer already had to invent as a wrapper (see 2.9): every list-shaped response returns `totalCount` , `returnedCount` , `offset` , `limit` , `hasMore` , with a sane default `limit` (recommend 50) and a hard max (recommend 500) enforced server-side regardless of what a client requests. `totalCount` must be present on every page (not just the first) so a consumer can answer "how many total" from any single call, matching the rationale already proven out in our own `listAssetsPaged` . This should apply

retroactively to any endpoint eWater chooses to version into `v2` — not just the new endpoints proposed in Section 3.

5.4 Deprecation policy

Recommend a `Sunset` HTTP header (RFC 8594) on any `v1` /unversioned endpoint once a `v2` equivalent ships, plus a maintained changelog entry per breaking change. Minimum deprecation window recommendation: 6 months from `Sunset` header appearing to actual removal, given that consumer integrations (like ours) are typically maintained intermittently, not continuously monitored for upstream API changes.

5.5 OpenAPI as the contract

Recommend eWater publish (or at minimum maintain internally, even if not yet public) an OpenAPI spec for the `v2` surface, mirroring the contract-first discipline our own project already follows (`lib/api-spec/openapi.yaml` → generated Zod validators + typed client hooks via Orval). This lets any consumer — including a future revision of our own MCP server — generate a typed client instead of hand-decoding response shapes from documentation prose, which is a meaningful source of the exact kind of quiet misinterpretation bugs already seen in the current integration (e.g. our own confirmed history of misreading `AssetFlowStatus` field names on the first pass — see our internal notes on `*AverageFlowRate` naming — a generated, typed client would have caught that at compile time instead of silently returning `null`).

Appendix A — Data caveats referenced throughout (source: Power BI data dictionary)

- **July 2025 "Data Waterfall" cutover:** `state.public.ztrilist_generalwateruse_*` tables are only reliably populated from ~July 2025 onward; pre-cutover analysis needs a legacy-table fallback, not a silent zero.
- **eSense exclusion/spike rules:** exclude `purpose = 'eSense'` from any disbursement total; cap supply-side eSense readings at 300 L/min before aggregating.
- **supertapposition is unreliable** for topup delivery status — use `tagtopupstate.state` instead.
- **Hardcoded currency/demographic constants** (`0.2109` KES/credit, `4.7` people/household) are Kenya-specific and wrong for every other country — use `ewater_central."Country" / "Currency"` instead.
- **Known-bad config row:** Austria maps to GMD at rate `1.0` in `ewater_central` — treat as test/junk data, not a real country config.
- **Sentinel placeholder timestamps:** ~0.003% of `generalwateruse_00f6` rows carry `2000-01-01` / `2086-01-01` bounds — filter these out of any date-sensitive query.
- **Supply (All)** is hardcoded to 3 assets (2015, 2211, 2105) — not a general fleet-wide bulk-supply table; NRW analysis should report `dataQuality:`
`"supply_partial" / "supply_unavailable"` for systems without dedicated instrumentation.

Appendix B — PII flag summary

Fields that may carry personally-identifiable information if eWater's schema joins them in, flagged here for eWater's own policy determination (not resolved by this document): tag-owner identity/name/phone on topup and tag-reconciliation responses (3.13–3.15), precise asset GPS coordinates when correlated with a named household, and any future join from `nfcid` to a real-world identity in the tag-flashing endpoints (3.11–3.12).